

School Pickup

Resumen de tecnologías por modulo

Documento orientativo sobre lenguajes, frameworks clave y la forma en que se comunican la web de direccion, la app de padres, la TV de salon y el servidor central. Incluye el mapa en vivo con Google Maps en la TV.

Proyecto escolar · Arquitectura en cuatro piezas

1. Vision general

School Pickup conecta a la escuela, a los padres y al salon en tiempo de salida. No hay un enlace directo permanente entre la web, el celular y la TV: todos hablan con un **servidor API** intermedio. Ese servidor guarda el padron, las zonas de entrega, los avisos de recogida y la ubicacion del padre; cada cliente consulta o publica ahi.

En la practica: la web administra y sincroniza datos; el movil avisa y comparte GPS; la TV muestra el panel del salon y el mapa en vivo del padre. La pieza comun es el backend en la nube.

1.1 Como se conectan entre si

Desde	Hacia	Como se habla
Web (admin)	Servidor API	Peticiones HTTP / JSON: sincroniza padron, grupos, zonas y pantallas. La web no habla directo con el celular ni con la TV.
Mobile (padres)	Servidor API	HTTP / JSON: notifica recogida, manda ubicacion GPS, marca llegada o recogido. Lee el estado de sus hijos.
TV (salon)	Servidor API	HTTP / JSON en consulta periodica (~1 s): lee alumnos, ETA, GPS y acciones. Envia Preparando / Preparado. El mapa Google usa aparte Maps/Directions.
Servidor API	Todos	Punto central. Persiste datos y expone rutas REST. No usa WebSocket: TV y movil se actualizan consultando el estado.

*Resumen: **web** ↔ **API** ↔ **mobile** y **web** ↔ **API** ↔ **TV**. Mobile y TV no se emparejan entre ellos; comparten el mismo estado en el servidor. El mapa Google en TV solo dibuja las coordenadas que ya vienen del API.*

2. Servidor API (el puente)

- **Lenguaje:** Kotlin (JVM).
- **Framework de servidor:** Ktor — rutas HTTP (avisos, ubicacion, estado de TV, sincronizacion).
- **Formato:** JSON.
- **Base de datos:** SQLite.
- **Despliegue:** Fly.io (HTTPS).

Guarda lat/lng del padre, destino de zona, calcula distancia/ETA (Haversine) y expone eso a la TV para el mapa en vivo.

3. Modulo Web (Direccion / admin)

- **Lenguajes:** TypeScript / JavaScript.
- **UI:** React + Vite + Tailwind CSS.
- **Mapas de zonas:** Leaflet + OpenStreetMap + Nominatim (buscar/marcar salidas).
- **Hosting:** Vercel; apunta al API.

Configura padron y zonas con GPS; al sincronizar, mobile y TV usan esos mismos puntos de entrega.

4. Modulo Mobile (app de padres)

- **Lenguaje:** Kotlin.
- **UI:** Jetpack Compose.
- **Cliente HTTP:** OkHttp hacia el API.
- **Ubicacion:** GPS nativo de Android (LocationManager) + servicio en primer plano durante la recogida.
- **Distancia / ETA:** Haversine (tambien en servidor). **No usa Google Maps SDK:** solo manda coordenadas.

El padre notifica recogida, opcionalmente comparte GPS; esa ubicacion es la que la TV pinta en Google Maps.

5. Modulo TV (panel de salon)

- **Lenguaje:** Kotlin.
- **UI:** Jetpack Compose (Android TV / pantallas grandes).
- **Cliente HTTP:** OkHttp — estado del salon y acciones del maestro.
- **Mapa en vivo (nuevo):** Google Maps SDK for Android — calles reales, pin del padre y de la zona.
- **Ruta sugerida (nuevo):** Google Directions API — polilinea azul padre → zona de salida.
- **API key:** se configura en local (no se sube al repositorio); requiere Maps SDK + Directions habilitados en Google Cloud.

La TV consulta el API cada segundo. Al tocar un alumno con GPS, abre el mapa Google: el pin azul se actualiza con la ubicacion en vivo y el encuadre muestra padre + zona (se acerca visualmente conforme el padre se acerca).

6. Frameworks y piezas clave (que hacen)

Solo las piezas que definen ubicacion, comunicacion, mapas y UI — no el inventario completo de librerias.

Pieza	Donde	Para que sirve
Ktor	Servidor	API HTTP compartida por web, mobile y TV.
OkHttp	Mobile y TV	Cliente HTTP al API; en TV tambien llama a Directions.
HTTP del navegador	Web	Sincroniza padron/zonas hacia el mismo API.
Jetpack Compose	Mobile y TV	Interfaz Android (app padres y panel TV).
React	Web	Consola de Direccion en el navegador.
LocationManager (GPS)	Mobile	Ubicacion real del padre; se publica al API en vivo.
Haversine (Geo)	Mobile y Servidor	Distancia en linea recta padre↔zona y ETA aproximado.
Leaflet + OSM / Nominatim	Web	Marcar y buscar puntos de salida en admin.
Google Maps SDK	TV	Mapa real en el salon: pin del padre en tiempo casi real y pin de zona.
Google Directions API	TV	Ruta de manejo sugerida (linea) entre padre y zona de entrega.
JSON / Serialization	Kotlin	Mismo formato de datos entre mobile, TV y servidor.
SQLite	Servidor	Persistencia del padron, pickups e historial.
Coroutines	Mobile, TV, Servidor	Consultas periodicas y red sin congelar la UI.

7. Flujo tipico de una recogida

1. Direccion marca zonas (GPS) en la web y sincroniza el padron al API.
2. El padre notifica recogida y comparte ubicacion desde el movil al API.
3. El servidor guarda coords, destino de zona y ETA (Haversine).
4. La TV muestra el alumno; al abrirlo, Google Maps pinta padre, zona y ruta Directions.
5. El maestro marca Preparando / Preparado; el padre marca llegada/recogido — todo via el API.

8. Nota sobre Google Maps (TV)

Maps SDK y Directions se usan **solo en la TV**. El movil no necesita Google Maps para calcular calles: basta con enviar lat/lng. La key se guarda en configuracion local del desarrollador (local.properties) y no debe subirse al repositorio publico. En Google Cloud deben estar habilitadas facturacion, Maps SDK for Android y Directions API.